Hardwired Control Unit Design

Week 6 — How Hardware Generates the Control-Step Sequence

Dr. Auqib Hamid Lone
Assistant Professor in Computer Science & Engineering

PCC CS-401: Computer Organization and Architecture

Department of Computer Science & Engineering
Government College of Engineering & Technology (GCET), Ganderbal

August 8, 2026

Where We Left Off (Week 5)

Last week: hardware datapaths, traced by hand.

- **Addressing modes** compute the effective address `EA` of an operand from an instruction's address field.
- A **single-bus** datapath moves `R1` $\leftarrow$ `R2+R3` in **3** control steps (T_1, T_2, T_3), using hidden registers `Y`, `Z` to buffer the bus.
- A **three-bus** datapath does the same transfer in **1** control step — more wiring, fewer cycles.

The gap we left open

We *wrote* T_1, T_2, T_3 ourselves, on paper. Nothing in the hardware yet decides "it is now T_2" or "assert Z_{in} this cycle." Something inside the CPU must do that, automatically, for every instruction. That is this week.

Roadmap: Three Questions, One Thread

1. **Control unit functions** — what does the control unit read, and what does it decide, every clock cycle?
2. **Generating the control-step sequence** — what hardware produces $T_1, T_2, \ldots$ automatically, instead of us writing it by hand?
3. **Hardwired control unit design** — given an instruction's opcode and the current step, how do we derive the actual logic gates that assert the right signals?

Running thread for today: we reuse the *same* instruction from Week 5, `R1` $\leftarrow$ `R2` + `R3` on the single-bus datapath, and build the actual control unit that drives it — so "the hardware decides" becomes gates and equations, not hand-waving.

Today: what generates the control-step sequence, automatically?

A Question We Skipped

Socratic question

In Week 5's trace, Y loaded from the bus at T_1 and Z loaded from the ALU at T_2 — but never the other way around. *Who decided that?* Not the ALU, not the bus, not the register file — none of them "know" what step it is.

Every one of those components is passive: it only moves data when *told* to. Something else watches the instruction and the clock, and tells every other component what to do, every cycle. That something is the **control unit**.

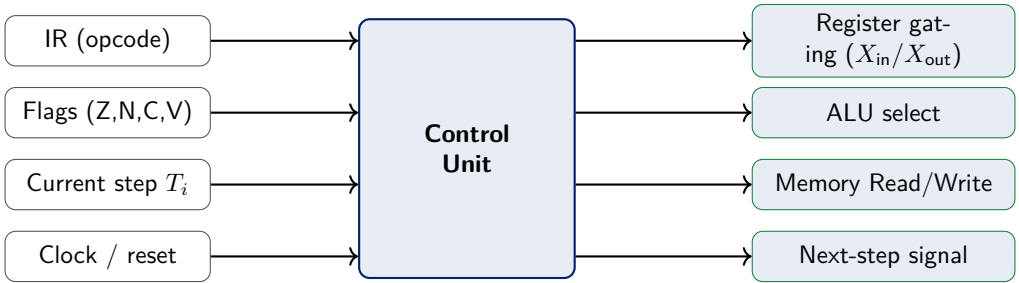
Definition: Control Unit

Control unit

The **control unit** is the hardware that, every clock cycle, examines the current instruction and the current control step, and **asserts the control signals** that make every other component (registers, ALU, memory, buses) do the right thing for that instruction at that step.

- It does not compute results itself — the ALU does that.
- It does not store data itself — registers and memory do that.
- It only decides *when* and *which* of those components act.

Control Unit: Inputs and Outputs



[1]

Notation: Register-Gating Control Signals

X_{in} / X_{out} notation (added to the course registry)

- $X_{\text{in}} = 1$ means: gate register `X` to *load* whatever is currently on the bus, this cycle.
- $X_{\text{out}} = 1$ means: gate register `X` to *drive* its contents *onto* the bus, this cycle.

Applied to Week 5's single-bus trace of `R1` ← `R2+R3`:

- T_1: $R2_{\text{out}}, Y_{\text{in}}$ (R2 drives the bus, Y latches it)
- T_2: $R3_{\text{out}}$, ALU ← add, Z_{in} (R3 drives the bus, ALU adds `[Y]`, Z latches the result)
- T_3: $Z_{\text{out}}, R1_{\text{in}}$ (Z drives the bus, R1 latches it)

These are exactly the control signals the control unit must assert — we now build the hardware that asserts them automatically.

Two Philosophies for Building a Control Unit

	Hardwired control	**Microprogrammed control**
Where the (opcode, step) $\rightarrow$ signal mapping lives	directly in logic gates	stored as microinstructions in a small memory
Speed per control step	fastest (pure combinational)	slightly slower (a memory read)
Ease of change	rigid — rewiring gates	easy — write new rows
Typical use	RISC, speed-critical	complex ISAs, flexibility-critical

This week vs. next week

Today we build **hardwired** control — the classical, gate-level approach. Week 7 rebuilds the *same* control unit as **microprogrammed** control, and we compare them directly.

Now: how is $T_1, T_2, \ldots$ generated — automatically?

Recall: We Wrote T_1, T_2, T_3 by Hand

Step	RTL	On the bus	What happens
T_1	$Y \leftarrow [R2]$	$[R2]$	R2 onto bus, latched into Y
T_2	$Z \leftarrow [Y] + [R3]$	$[R3]$	R3 onto bus $\rightarrow$ ALU; ALU adds Y; result $\rightarrow Z$
T_3	$R1 \leftarrow [Z]$	$[Z]$	Z onto bus, latched into R1

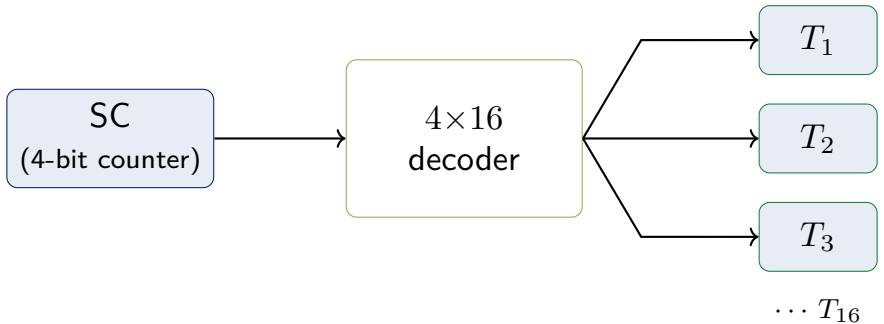
Socratic question

Nothing in this table says *how* the hardware knows it is T_2 and not T_1. What physical component keeps track of "which step we are on"?

The Sequence Counter (SC)

New notation: SC

The **sequence counter (SC)** is a small counter register that holds the number of the current control step. A **step decoder** converts SC's binary value into one-hot signals $T_1, T_2, T_3, \ldots$ — exactly one T_i is active per cycle.



[2] — cf. Fig. 5-6, p.137 (the sequence-counter/timing-decoder portion of the control unit). Shown here as the general ISA-sized counter (4 bits, up to 16 steps); this course's worked trace only uses T_1, T_2, T_3, so its SC can be sized down to 2 bits — see "The Two Threads Meet."

How SC Advances

- Every clock pulse, by default: $SC \leftarrow [SC] + 1$ — the counter just counts up, and the step decoder's one-hot output moves to the next T_i.
- At the *last* step of an instruction, the control unit instead asserts SC_{clear}: $SC \leftarrow 0$, which returns the decoder to T_1 — signaling "this instruction is done; begin fetching the next one."
- Without SC_{clear}, the counter would keep counting forever with no instruction ever finishing.

Applied to our trace

For `R1` $\leftarrow$ `R2+R3`, SC_{clear} fires at T_3 — the last step of this instruction.

Worked Trace Revisited: Where SC Points

Step	SC value	Active line	RTL
T_1	0	$T_1 = 1$	$Y \leftarrow [R2]$
T_2	1	$T_2 = 1$	$Z \leftarrow [Y] + [R3]$
T_3	2	$T_3 = 1$, $SC_{\text{clear}} = 1$	$R1 \leftarrow [Z]$, then $SC \leftarrow 0$

This is the piece Week 5 was missing: SC plus the step decoder *is* the hardware that turns "now it's T_2" from something we wrote by hand into something the circuit does on its own, every cycle.

Socratic Check: What Happens After T_3?

Question

Suppose we forgot to wire up SC_{clear}. What would the hardware do after T_3?

- SC would keep incrementing: $T_4, T_5, \ldots$ forever.
- The step decoder would produce one-hot lines for steps that *no instruction defines any signals for* — the datapath would simply idle, and the next instruction would never be fetched.
- SC_{clear} is not optional bookkeeping — it is what makes the control unit cycle through instructions at all, rather than freezing after the first one.

But different instructions need different micro-operations at each step — how does hardware know which?

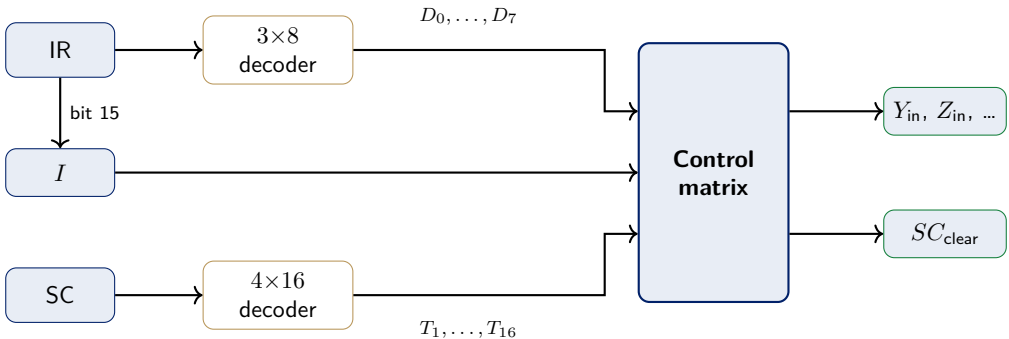
The Missing Piece: Opcode Decoding

Socratic question

SC tells the control unit *when* (which T_i). But Y_{in} should fire at T_1 for an ADD, while a LOAD instruction needs something completely different at its T_1. SC alone cannot know *which* instruction is running.

- New notation: D_i — a one-hot **opcode decoder** output, where $D_{\text{ADD}} = 1$ exactly when the IR holds the ADD opcode (and every other $D_j = 0$).
- New notation: the I **flip-flop** — captures IR bit 15, which marks an *indirect* addressing instruction (Week 5). When $I = 1$, signals that resolve an address indirectly must behave differently, so the control matrix receives I as an extra input.
- Every control signal will turn out to depend on *both* an opcode line (D_i) and a step line (T_j) — never one alone.

Two Decoders Feed the Control Matrix



Every control signal is a Boolean function of the D_i lines, the T_j lines, and (for indirect-addressing-sensitive signals) the I flip-flop together — `IR` bits 0–11 also feed the control logic gates directly (omitted above for clarity) — cf. Mano, Fig. 5-6, p.137 [2].

Building the State Table for R1 ← R2+R3

Step	RTL	Control signals active
T_1	$Y \leftarrow [R2]$	$R2_{\text{out}}, Y_{\text{in}}$
T_2	$Z \leftarrow [Y] + [R3]$	$R3_{\text{out}}$, ALU=add, Z_{in}
T_3	$R1 \leftarrow [Z]$	$Z_{\text{out}}, R1_{\text{in}}, SC_{\text{clear}}$

This is the state table

Every signal in the "active" column is only asserted when *both* $D_{\text{ADD}} = 1$ *and* the listed $T_i = 1$. The state table is the bridge between the RTL trace we already know and the Boolean equations we derive next.

From State Table to Boolean Equations

Reading straight off the state table for `ADD` (opcode line D_{ADD}):

$$
\begin{aligned}
Y_{\text{in}} &= D_{\text{ADD}} \cdot T_1 \\
Z_{\text{in}} &= D_{\text{ADD}} \cdot T_2 \\
R1_{\text{in}} &= D_{\text{ADD}} \cdot T_3 \\
SC_{\text{clear}} &= D_{\text{ADD}} \cdot T_3
\end{aligned}
$$

- Each equation is just: "this signal fires when this instruction is running *and* we are at this step."
- If a *different* instruction also needs Z_{in} at some step, its term gets **OR**'d into the same equation — covered next.

Worked Example: Deriving Z_{in} by Hand

Suppose SUB also uses Z to hold its result, at its own T_2

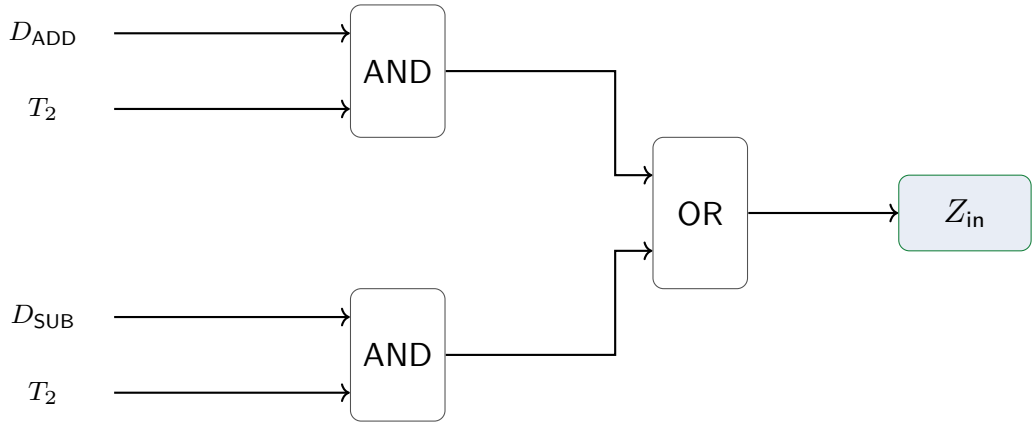
SUB has an identical 3-step shape to ADD, differing only in the ALU function selected at T_2.

1. From the ADD state table: Z_{in} fires at $D_{\text{ADD}} \cdot T_2$.
2. From the SUB state table: Z_{in} also fires at $D_{\text{SUB}} \cdot T_2$.
3. **Any** instruction whose state table lists Z_{in} at T_2 contributes an OR term.

$$Z_{\text{in}} = (D_{\text{ADD}} \cdot T_2) + (D_{\text{SUB}} \cdot T_2) = T_2 \cdot (D_{\text{ADD}} + D_{\text{SUB}})$$

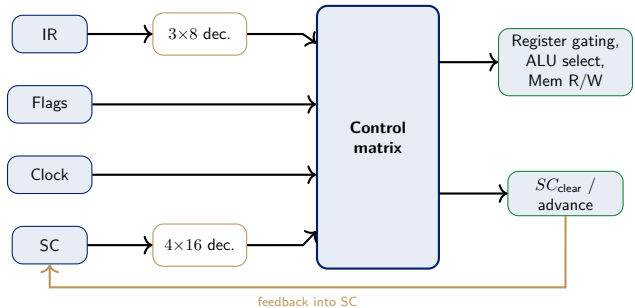
This is the general pattern: every control signal is an OR, across all instructions that use it, of (opcode line · step line).

The Control Matrix as Actual Logic Gates



Two AND gates (one per instruction that needs Z_{in} at T_2), one OR gate combining them — this is literally what "hardwired" means.

Full Hardwired Control Unit Block Diagram



cf. Mano, Fig. 5-6, p.137 (core: two decoders + SC + control logic gates), extended with explicit Flags/Clock inputs and the SC feedback loop [2, 1].

Socratic Check: Adding a New Instruction

Question

Suppose we now add a `MUL` instruction that also uses Z at its own T_2. What has to change in the hardwired control unit we just built?

- The opcode decoder needs a *new output line*, D_{MUL}.
- Every equation for a signal `MUL` needs (e.g. Z_{in}) must be **re-derived**, and a new AND gate ($D_{\text{MUL}} \cdot T_2$) must be physically wired into the existing OR gate.
- This is not a software change — it is new gates, on the actual chip, for *every* signal the new instruction touches.

Fast, but rigid — what does that cost us?

Hardwired Control: Trade-offs

Cost vs. flexibility

- Pro: pure combinational logic — control signals appear the *same cycle*, with no extra memory-access delay.
- Pro: no wasted circuitry — the gates that exist are exactly the ones the ISA needs, nothing more.
- Con: adding, removing, or fixing a bug in one instruction means re-deriving equations and *physically rewiring* logic gates.
- Con: the control matrix grows with the number of instructions $\times$ the number of signals — verifying it by hand gets harder fast.

Where This Breaks Down

- Our worked example had *one* instruction (`ADD`), 3 steps, and a handful of signals — the equations fit on one slide.
- A real ISA has dozens to hundreds of instructions, each with its own state table, each contributing AND terms to *every* shared signal's OR equation.
- Every new instruction, every microarchitectural bug fix, means re-deriving Boolean equations and re-wiring gates by hand — design and verification cost explodes with ISA complexity [3].

Bridge to Week 7

A different idea

What if, instead of wiring $Y_{\text{in}}, Z_{\text{in}}, R1_{\text{in}}, \ldots$ directly into logic gates, we simply *wrote them down* — one row per control step, stored in a small memory — and let a counter step through the rows?

- Adding a new instruction becomes: write new rows into memory, not redesign gates.
- The trade-off flips: slightly slower per step (a memory read), far easier to design, verify, and modify.
- That is **microprogrammed control** — next week, we rebuild *this same* `R1` $\leftarrow$ `R2+R3` control unit that way.

The Two Threads Meet

	Single bus	**Three bus**
Control steps for `R1` ← `R2+R3`	3 (T_1, T_2, T_3)	1 (T_1)
`SC` width	2 bits for this trace (general: up to 4 bits / 16 steps, cf. the SC frame)	barely needed
Step-decoder outputs	3	1
State-table rows for `ADD`	3	1
Control-matrix cost	more AND/OR gates	fewer gates, more bus wiring

- Fewer control steps means a *smaller* state table and fewer AND/OR gates in the control matrix — the cost moves from control logic into bus wiring instead.
- Neither the datapath choice (Week 5) nor the control unit choice (Week 6) is free — every design decision moves cost somewhere else on the chip.

Summary

- **Control unit** = the hardware that reads (opcode, step, flags) and asserts control signals for every other component, every cycle.
- **Sequence counter (SC)** + step decoder generate $T_1, T_2, \ldots$ automatically; SC_{clear} resets to T_1 at an instruction's last step.
- **Opcode decoder** produces one-hot D_i lines; every control signal is an OR, across instructions, of $(D_i \cdot T_j)$ terms — the **state table** makes this systematic.
- **Hardwired control**: state table $\rightarrow$ Boolean equations $\rightarrow$ AND/OR gates. Fast, but rigid — new instructions mean new gates, wired by hand.
- This rigidity is exactly what motivates **microprogrammed control** (Week 7): store control words in memory instead of wiring them into gates.

References


V. Carl Hamacher, Zvonko G. Vranesic, and Safwat G. Zaky.
Computer Organization.
McGraw-Hill, 5th edition, 2002.


M. Morris Mano.
Computer System Architecture.
Prentice-Hall, 3rd edition, 1993.


William Stallings.
Computer Organization and Architecture: Designing for Performance.
Pearson, 10th edition, 2015.